



DevOps Capstone Project: Todo List Application

Team (244) Members:

- Soha Ashraf Eltokhy.
- Haneen Sabry Abdelazeem.
- Ali Tarek Ali.
- Michael Fayez Beshara.

Github Repo(with 2 Branches): <https://github.com/nhahub/NHA-244>

Architectural Overview & Project Phases



Application & Containerization

A Node.js Todo List application (Express.js + MongoDB) is containerized with Docker, using multi-stage builds.



Orchestration & Automation

Kubernetes (minikube) orchestrates the application, while Ansible automates VM provisioning.



CI/CD & Registry

GitHub Actions manages the CI/CD pipeline, pushing images to Docker Hub.



Infrastructure

The application is deployed on an Ubuntu VM.

Project Phases: A Structured Approach

01

Phase 1: Application Dockerisation

The application was Dockerised using a multi-stage Dockerfile. Images were built, tested, and pushed to Docker Hub.

03

Phase 3: Kubernetes Deployment

Kubernetes manifests were crafted and the application deployed with 2 replicas. It was exposed via NodePort.

02

Phase 2: VM Automation with Ansible

Ansible automated VM provisioning, installing Docker, kubectl, minikube, and MongoDB.

04

Phase 4: CI/CD Pipeline

A GitHub Actions workflow was established for automated Docker image builds. This integrated with Docker Hub using a multi-tag strategy.

Triggering the CI/CD Pipeline: Live Demo

A light gray circular icon containing the code symbols `</>`.

Step 1: Make a Code Change

Navigate to the application repository: `cd ~/Desktop/capstone-project/ToDo-List-nodejs`. Edit the homepage file `views/home.ejs`, for instance, by modifying line 27 to "Todos - Live Demo".

A light gray circular icon containing the word `git` in a bold, lowercase font.

Step 2: Commit and Push Changes

Stage the changes with `git add views/home.ejs`, then commit with a descriptive message like "Demo: Update homepage for presentation". Finally, push to GitHub using `git push origin master` to automatically trigger the CI/CD pipeline.

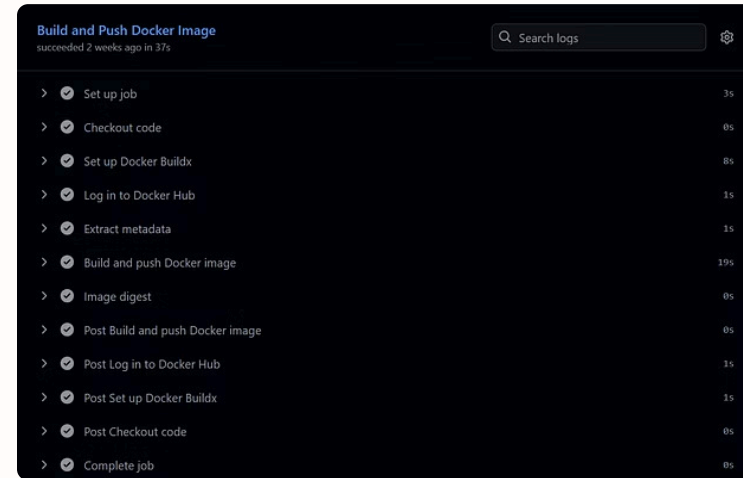
Monitoring the CI/CD Pipeline and Image Verification

Step 3: Monitor CI/CD Pipeline

The pipeline should show the following stages, expected to complete within 40-60 seconds:

- Checkout code
- Set up Docker Buildx
- Login to Docker Hub
- Build and push Docker image
- Image tagged with latest and master-

<https://github.com/alitarek-dot/ToDo-List-nodejs/actions>



The screenshot shows a GitHub Actions workflow log for the job 'Build and Push Docker Image'. The log indicates the job succeeded 2 weeks ago in 37s. A search bar is visible at the top right. The log entries are as follows:

Step	Duration
> Set up job	3s
> Checkout code	0s
> Set up Docker Buildx	0s
> Log in to Docker Hub	1s
> Extract metadata	1s
> Build and push Docker image	19s
> Image digest	0s
> Post Build and push Docker image	0s
> Post Log in to Docker Hub	1s
> Post Set up Docker Buildx	1s
> Post Checkout code	0s
> Complete job	0s

Step 4: Verify New Image on Docker Hub

Check Docker Hub to confirm the new image has been pushed. Look for the master- tag; its timestamp should align with push time.

<https://hub.docker.com/r/alitarek123/todo-list-nodejs/tags>

Deploying to Kubernetes & Verifying Changes



Step 5: Deploy Updated Image to Kubernetes

SSH into the VM: `ssh -i ~/.ssh/vm_ansible alitarek@192.168.74.128`. Then, update the Kubernetes deployment using the new image tag. Monitor the rollout status to ensure successful deployment and verify pods are running with the new image.



Step 6: Verify Changes in Browser

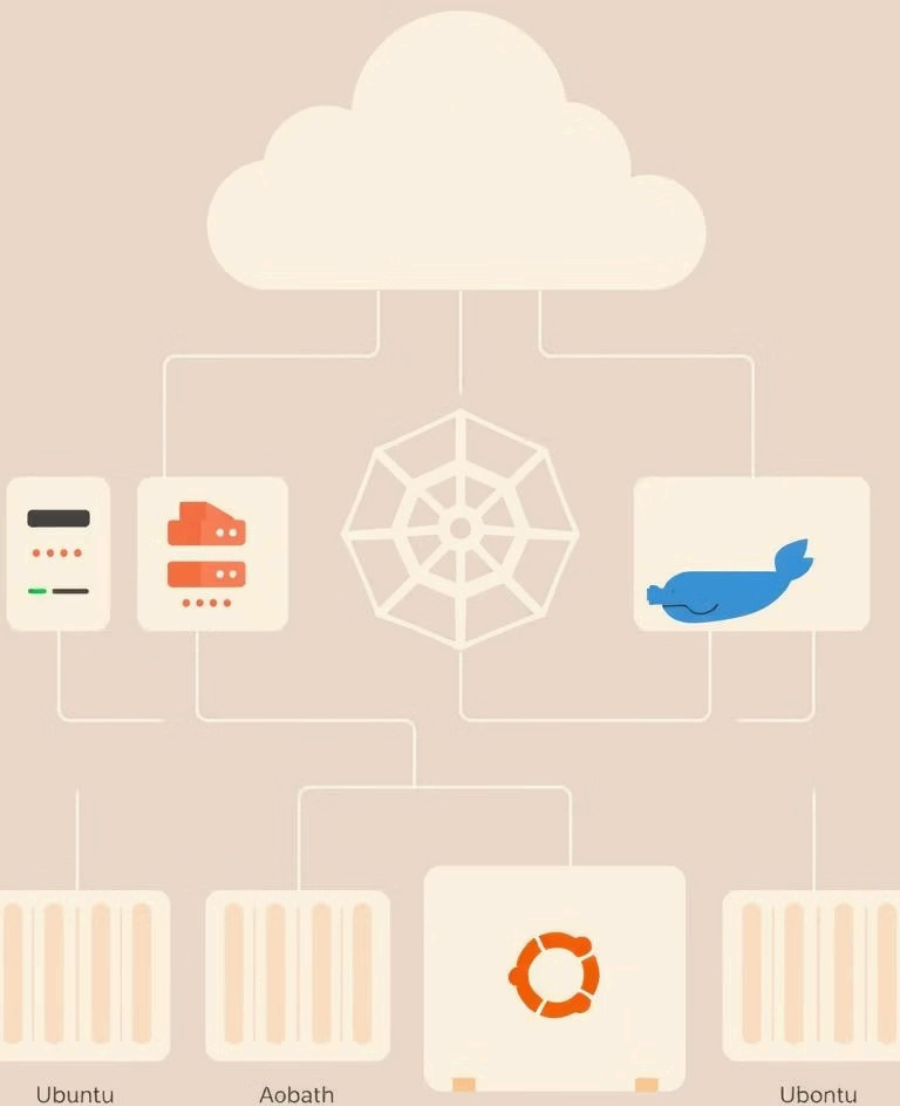
Access the application at <http://192.168.74.128:30080>. The homepage should now display your updated text. A quick browser refresh might be needed to see the changes.

Accessing the Application

The primary access method is via the URL: <http://192.168.74.128:30080>

Verification Commands

- **Check pod status:** `kubectl get pods -n todo-app`
- **Check service details:** `kubectl get svc -n todo-app`
- **View application logs:** `kubectl logs -n todo-app -l app=todo-app --tail=50`
- **Test from inside minikube:** `docker exec minikube curl -s http://10.244.0.3:4000 | grep ""`



Key Metrics & Details

Infrastructure Specifications

- **VM OS:** Ubuntu 24.04.3 LTS
- **VM IP:** 192.168.74.128
- **Minikube IP:** 192.168.49.2
- **Kubernetes Version:** v1.34.0
- **Container Runtime:** Docker
- **Minikube Resources:** 2 CPUs, 2GB RAM

Application Details

- **Replicas:** 2 pods for high availability
- **Container Port:** 4000
- **Service Type:** NodePort (30080)
- **Health Checks:** Liveness and readiness probes configured
- **Resource Limits:** CPU and memory constraints applied

Docker Image Details

- **Repository:** alitarek123/todo-list-nodejs
- **Image Size:** ~209MB (optimized multi-stage build)
- **Base Image:** node:18-slim
- **Architecture:** linux/amd64
- **Latest Tag:** master-7048b27

MongoDB Configuration

- **Deployment:** Docker container on VM (outside Kubernetes)
- **Version:** MongoDB 7 (jammy)
- **Port:** 27017
- **Database:** todolistDb
- **Authentication:** Enabled with credentials stored in K8s secrets

Troubleshooting Guide

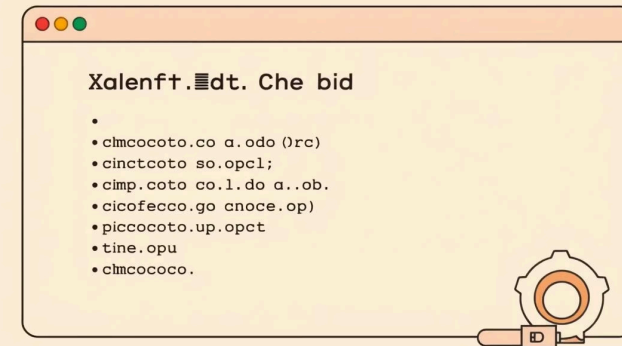
If Application is Not Accessible

- **Check minikube status:** `minikube status`
- **Restart minikube if needed:** `minikube delete && minikube start --driver=docker --cpus=2 --memory=2048 --force`
- **Reapply Kubernetes manifests:** Use `kubectl apply -f -` for all manifest files.
- **Wait for pods to be ready:** `kubectl wait --for=condition=ready pod -l app=todo-app -n todo-app --timeout=120s`



If CI/CD Pipeline Fails

- **Check GitHub Actions logs:** Review the failed workflow run details.
- **Common issues:** Docker Hub credentials expiration, build timeouts due to rate limits, or Dockerfile syntax errors.



Issues Faced & Solutions

Issue 1: Disk Space Critically Low

Problem: VM disk at 98% capacity, preventing minikube from starting.

Solution: Cleaned Docker system, removed minikube cache, trimmed journal logs, and cleaned apt cache, freeing ~1GB of disk space.

Issue 2: MongoDB Connectivity

Problem: Application pods couldn't connect to MongoDB on VM host due to network isolation.

Solution: Deployed MongoDB inside Kubernetes with PersistentVolumeClaim, Deployment, and ClusterIP Service, resolving cross-network communication issues.

Issue 3: Secret Key Mismatch

Problem: MongoDB pod failing due to incorrect secret key references.

Solution: Corrected environment variable references in `mongodb-deployment.yaml` to match Kubernetes secret keys.

Issue 4: External Application Access

Problem: Application accessible inside minikube but not from outside the VM.

Solution: Configured iptables NAT rules to forward VM IP traffic to minikube's NodePort, enabling external access.

Each challenge presented an opportunity to deepen our understanding of DevOps principles, from resource management to network configuration and secure credential handling. The iterative process of identifying, diagnosing, and resolving these issues was crucial to the project's success.

Security Considerations & Technologies Used

Implemented Security Measures

Container Hardening

Non-root container user, dropped Linux capabilities, and no privilege escalation allowed.

Authentication & Secrets

MongoDB authentication enabled, credentials stored securely in Kubernetes Secrets and GitHub secrets for CI/CD.

Infrastructure Security

SSH key-based authentication for Ansible automation, and resource limits to prevent exhaustion.

Technologies & Tools Used

Application	Node.js	18
	Express.js	Latest
Database	MongoDB	7-jammy
Containerisation	Docker	29.0.4
Orchestration	Kubernetes (minikube)	1.34.0
CLI Tools	kubectl	1.31.0
Automation	Ansible	Latest
CI/CD	GitHub Actions	Latest
Registry	Docker Hub	-
OS	Ubuntu	24.04.3 LTS

Security was a paramount consideration throughout the project, ensuring robust protection from code to deployment. Key technologies and tools underpin every aspect of this modern DevOps pipeline.

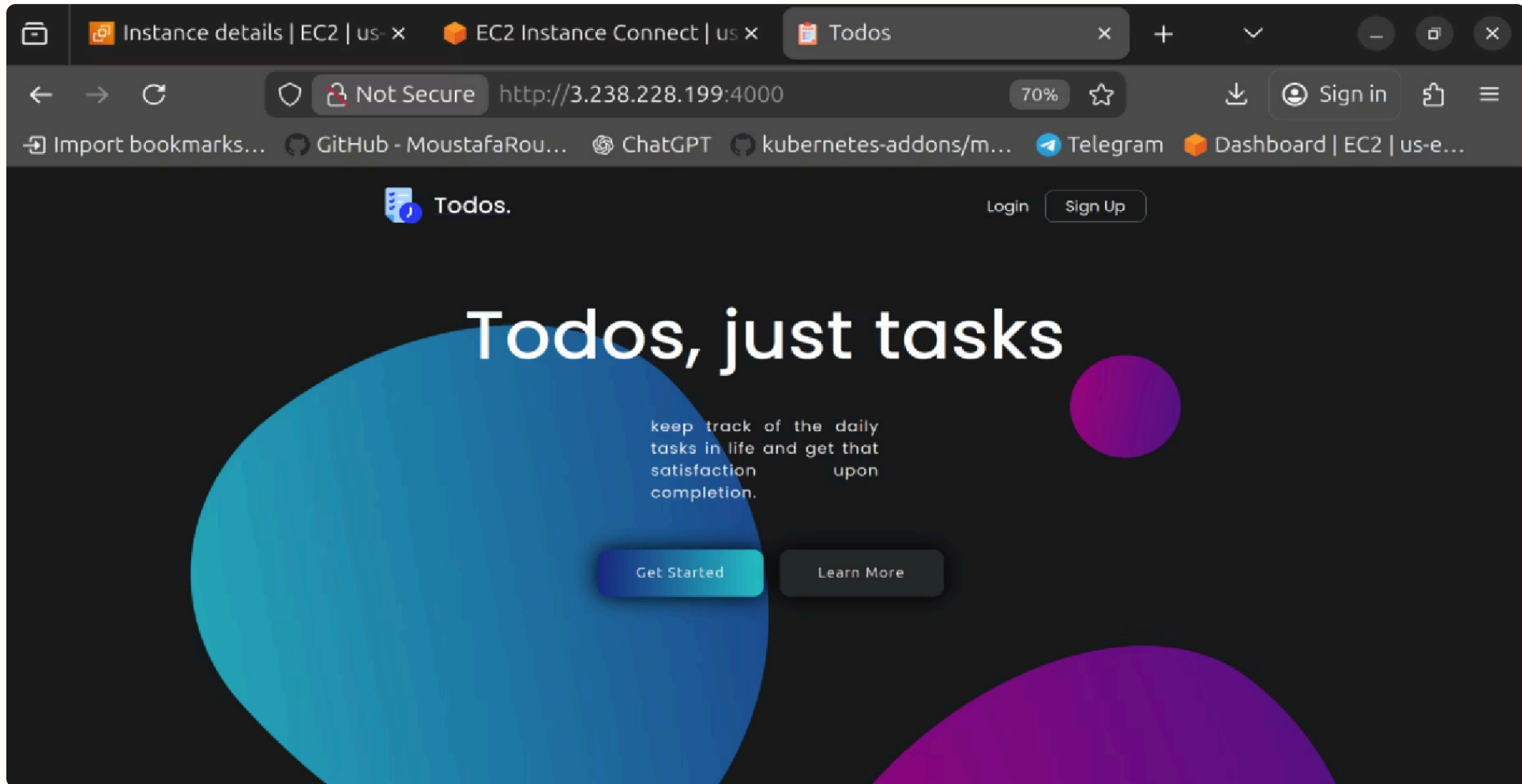
2nd Approach

Hosting the application on EC2 server "AWS cloud"

2nd Approach: Hosting the application on EC2 server "AWS cloud" Using Docker Compose

- Setup and Containerization. (Docker contentainers).
- Infrastructure Provisioning & Deployment. (Ansible Configurations).
- Monitoring, Visualization.(Prometheus and Granfana).

2nd Approach



2nd Approach

Docker Compose

```
[+] up 54/56
✓ Image grafana/grafana:12.1 Pulled 27.6s
✓ Image containrrr/watchtower Pulled 4.5s
✓ Image prom/prometheus:v3.5.0 Pulled 18.9s
✓ Image mongo:8 Pulled 29.2s
✓ Image quay.io/prometheus/node-exporter:latest Pulled 5.4s
✓ Image sohaash/todo_app:v0.2 Pulled 14.9s
✓ Network todos-nodejs-_monitoring Created 0.1s
✓ Network todos-nodejs-_default Created 0.0s
✓ Volume todos-nodejs-_mongo_data Created 0.0s
✓ Volume todos-nodejs-_prometheus_data Created 0.0s
✓ Volume todos-nodejs-_grafana_data Created 0.0s
✓ Container mongodb Created 0.4s
✓ Container todos-nodejs--watchtower-1 Created 0.4s
✓ Container prometheus Created 0.4s
✓ Container node_exporter Created 0.4s
✓ Container grafana Created 0.4s
```

2nd Approach

playbook.yaml

```
TASK [Gathering Facts] *****
[WARNING]: Platform linux on host 3.238.228.199 is using the discovered Python interpreter at
/usr/bin/python3.12, but future installation of another Python interpreter could change the meaning
of that path. See https://docs.ansible.com/ansible-
core/2.18/reference_appendices/interpreter_discovery.html for more information.
ok: [3.238.228.199]

TASK [Ensure ansible-user exists] *****
changed: [3.238.228.199]

TASK [Allow passwordless sudo for ansible-user] *****
ok: [3.238.228.199]

TASK [Create .ssh directory for ansible-user] *****
ok: [3.238.228.199]

TASK [Copy authorized_keys from ubuntu to ansible-user] *****
ok: [3.238.228.199]

TASK [Install dependencies for Docker] *****
ok: [3.238.228.199]

TASK [Add Docker GPG key] *****
ok: [3.238.228.199]

TASK [Add Docker repository] *****
ok: [3.238.228.199]
```

2nd Approach

Instance details | x

EC2 Instance Conn x

Prometheus Time x

NHA-244/Todos-n x

+

⌵

⌵

⌵

⌵

←

→

↻

⛔ Not Secure

http://3.238.228.199:9090/targets

70%

☆

⬇

👤 Sign in

🔖

☰

🔖 Import bookmarks...

🐙 GitHub - MoustafaRou...

🌀 ChatGPT

🐙 kubernetes-addons/m...

📠 Telegram

📦 Dashboard | EC2 | us-e...

🔥 Prometheus

🔍 Query

🔔 Alerts

Status > Target health ⌵

🌙

🗨 1

⚙

📖

node_exporter

1 / 1 up 🟢 ⌵

Endpoint	Labels	Last scrape	State
http://node_exporter:9100/metrics	instance="node_exporter:9100" job="node_exporter" ⌵	🕒 4.556s ago 🕒 24ms	UP

Discovered labels:

__address__="node_exporter:9100"

__metrics_path__="/metrics"

__scheme__="http"

__scrape_interval__="5s"

__scrape_timeout__="5s"

job="node_exporter"

prometheus

1 / 1 up 🟢 ⌵

Endpoint	Labels	Last scrape	State
http://prometheus:9090/metrics	instance="prometheus:9090" job="prometheus" ⌵	🕒 5.123s ago 🕒 38ms	UP

Discovered labels:

__address__="prometheus:9090"

__metrics_path__="/metrics"

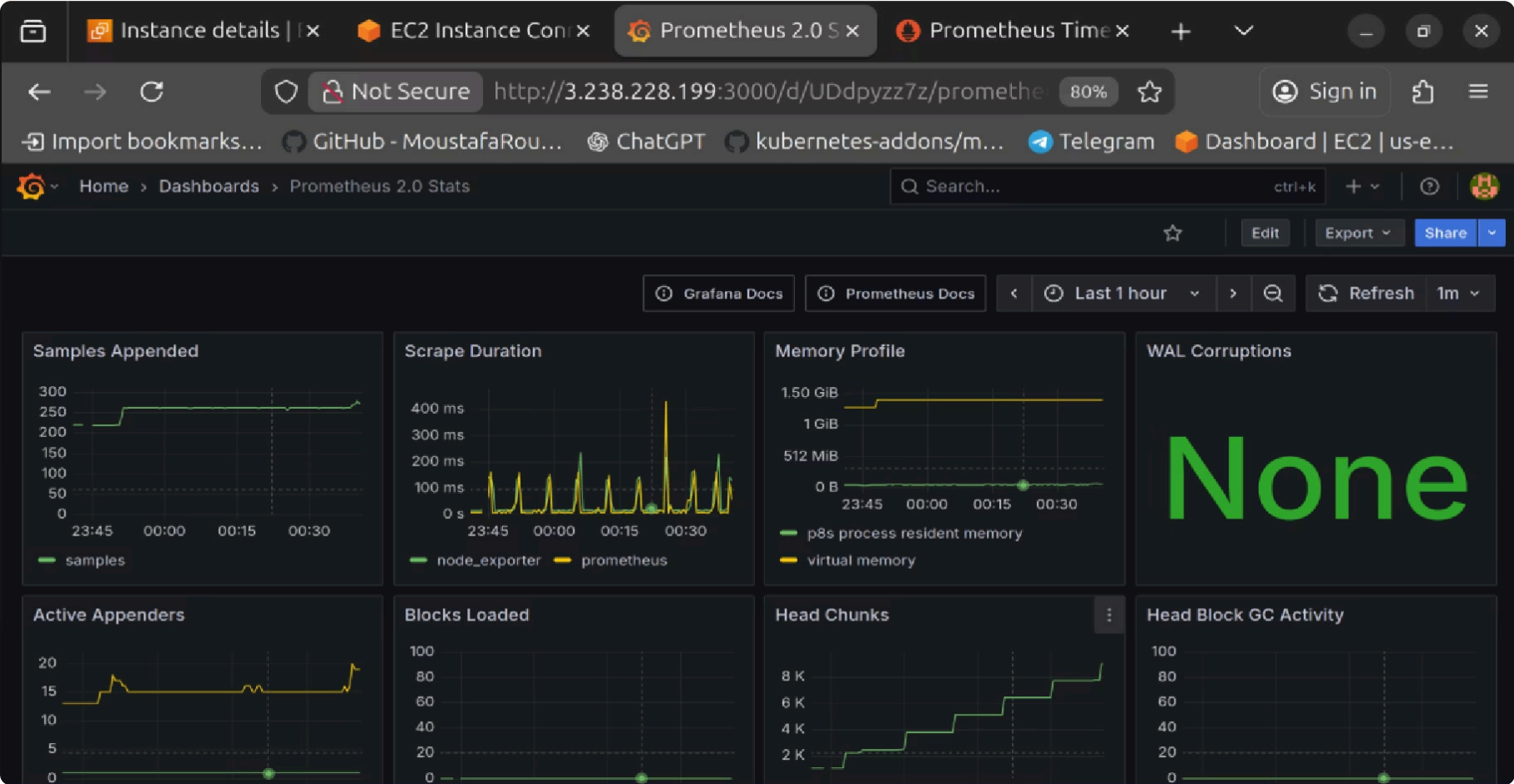
__scheme__="http"

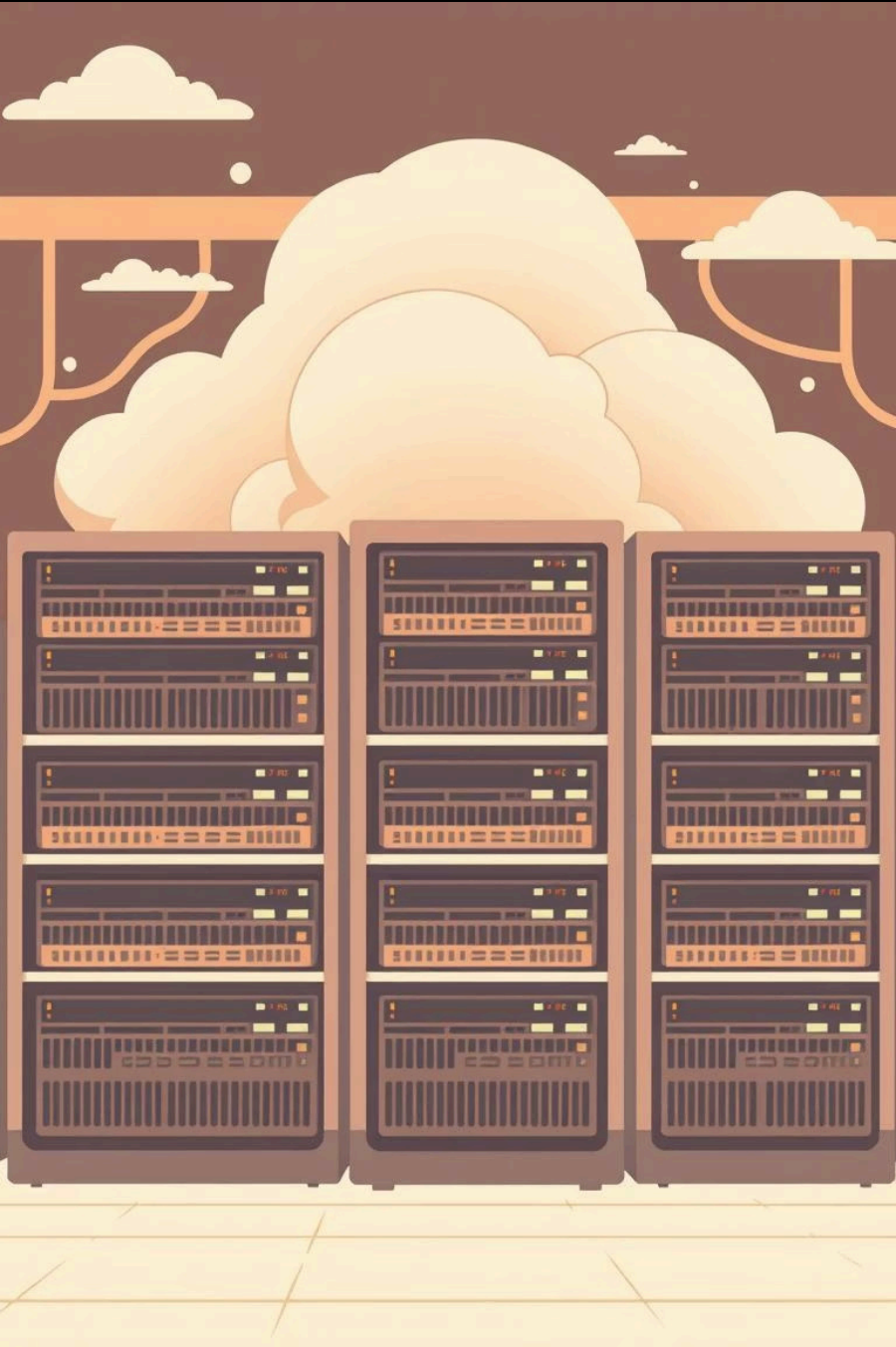
__scrape_interval__="5s"

__scrape_timeout__="5s"

job="prometheus"

2nd Approach





DevOps Capstone Project: Todo List Application

Thank You

Thank you for your attention. We appreciate you taking the time to learn about our project.